

link:"https://chatgpt.com/share/6a918a70-ea8c-83e8-8b44-2c9569f14609"

This is a copy of a shared ChatGPT chat. It won't be added to memory.

Report conversation

Absolutely. To review it against the actual assignment requirements, I need to see your implementation and the assignment specification.

Please upload either:

Assignment PDF/DOCX + your Django project ZIP, or

Paste the assignment requirements and your relevant code (models.py, serializers.py, views.py, recommendation/packing service, and tests).

I'll review it in this format:

Area Requirement Your implementation Status Product model Dimensions + weight — ☐/☐/☐ Box model Dimensions + capacity + cost — Orientation Check rotations — Multiple products Single-row packing — Recommendation Best box selection — API Request/response format — Validation Invalid input handling — Edge cases No suitable box, etc. — Tests Required scenarios — Documentation README/API docs — I'll specifically look for missing functionality, incorrect assumptions, algorithmic bugs, API design problems, Django/DRF issues, validation gaps, and anything likely to lose points in the hiring evaluation.

If you upload the project ZIP and assignment document together, I can do a much more precise requirement-by-requirement review.

Review the final project against the assignment requirements and provide a submission checklist.

Yes. Since you're asking for a final project review against the assignment, I need the actual final project and assignment specification in this conversation (or in your Library) to make a real requirement-by-requirement assessment.

Please upload the final Django project ZIP and the assignment PDF/DOCX. I'll check the implementation rather than guessing from the design we discussed earlier.

Submission checklist I'll use ☐ Functional requirements Product creation/input works

Box inventory works

Product dimensions are validated

Product weight is validated

Box dimensions are validated

Box weight capacity is enforced

Box cost is handled correctly

Product rotations/orientations are checked

Multiple products per order are supported

Quantities are handled correctly

Single-row packing constraint is correctly enforced

Best suitable box is selected

No-suitable-box case is handled cleanly

☐ API Correct endpoints

Correct HTTP methods

Request validation

Correct response format

Appropriate HTTP status codes

Useful error messages

Invalid/missing input handled

API doesn't expose unnecessary internal fields

☐ Algorithm All relevant orientations tested

Exact-fit dimensions accepted

Total row length calculated correctly

Total order weight calculated correctly

Box capacity enforced

Ranking/selection criteria are deterministic

Cost is considered according to the assignment

Algorithm is separated from API/view code

No accidental claim that deterministic packing logic is "AI"

☐ Django quality Clean project/app structure

Models use appropriate field types

DecimalField used appropriately for money

Database constraints/validators where appropriate

Serializers handle input validation

Business logic isn't buried in views

No hard-coded production data

Migrations included

Environment/configuration handled properly

☐ Testing Normal recommendation

Rotation case

Exact boundary fit

Product too large

Weight overflow

Multiple products

Multiple quantities

Row-length overflow

No suitable box

Invalid dimensions

Invalid quantity

Empty order

API-level tests

Tests are repeatable and isolated

☐ Submission quality README.md

Setup instructions

Dependency list / requirements.txt

Database setup instructions

Migration instructions

API documentation

Example requests/responses

Example test command

Clear explanation of packing assumptions

Clear explanation of recommendation criteria

No secrets/API keys committed

.gitignore included

No unnecessary files (___pycache___, local DB if prohibited, IDE files, etc.)

Final pre-submission test Run from a clean environment:

```
python -m venv venv pip install -r requirements.txt python manage.py migrate python manage.py test  
python manage.py check python manage.py runserver
```

 Then manually verify the main API flow:

Create/seed boxes ↓ Create/submit order ↓ Validate dimensions & weight ↓ Check product orientations ↓
Check single-row constraint ↓ Calculate eligible boxes ↓ Rank candidates ↓ Return recommended box
Upload the final ZIP + assignment document, and I can turn this into a concrete report such as “18/22
requirements satisfied; 2 bugs; 2 missing items”, with the exact files/functions that need fixing.

After I make the changes, help me verify that the implementation still satisfies the original assignment requirements.

Absolutely. After you make the changes, send me the updated project ZIP (and the original assignment document if it isn't still available in the conversation).

I'll do a final regression review against the original requirements, not just review the new code.

What I'll verify Requirement-by-requirement compliance

Map every original requirement to the implementation.

Mark each as ☐ Pass / ☐☐ Partial / ☐ Missing.

Box-fitting correctness

All relevant product orientations.

Exact dimension matches.

Products that cannot fit.

Single-row packing.

Multiple products and quantities.

Total weight/capacity.

Box availability and cost.

API behavior

Request validation.

Successful recommendation response.

Error responses.

HTTP status codes.

Edge cases.

Django quality

Models and relationships.

Serializers.

Views/API endpoints.

Service/business logic separation.

Database migrations.

Configuration/security issues.

Regression testing

Run/check the existing tests where possible.

Inspect whether your changes introduced inconsistencies.

Identify missing tests for newly changed functionality.

Submission readiness

README/setup instructions.

Dependencies.

API examples.

Test instructions.

Clean repository.

No secrets or unnecessary files.

Final report I'll give you a concise final verdict like:

ASSIGNMENT COMPLIANCE ===== Core requirements 9/9 ☐ API requirements 6/6
☐ Packing logic 5/5 ☐ Validation 4/5 ☐ Testing 8/8 ☐ Documentation 3/4 ☐

OVERALL: READY WITH 2 MINOR FIXES I'll also distinguish actual assignment requirements from optional improvements, so you don't waste time polishing things that aren't needed for the submission.

When you've made the changes, upload the updated project and I'll perform the final verification.